파이썬 OOP 개념

객체 지향 프로그래밍은 파이썬의 기본 개념으로, 개발자가 모듈식, 유지 관리 및 확장 가능한 애플리케이션을 구축할 수 있도록 지원합니다.

객체 지향 프로그래밍(OOP)은 객체와 클래스를 사용하여 실제 개체와 그 동작을 표현하는 코드를 구성하는 방식입니다. 객체 지향 프로그래밍에서 객체는 특정 데이터를 가진 속성을 가지며, 메서드를 사용하여 특정 동작을 수행할 수 있습니다.

파이썬의 OOP 개념

- 파이썬의 클래스
- 파이썬의 객체
- 파이썬의 다형성
- 파이썬에서의 캡슐화
- 파이썬에서의 상속
- 파이썬에서의 데이터 추상화

파이썬 클래스

클래스는 객체들의 집합입니다. 클래스는 객체를 생성하기 위한 청사진입니다. 클래스는 생성된 객체(인스턴스)가 가질 수 있는 속성과 메서드 집합을 정의합니다.

Python 클래스에 대한 몇 가지 사항:

- 클래스는 class 키워드로 생성됩니다.
- 속성은 클래스에 속하는 변수입니다.
- 속성은 항상 공개되어 있으며 점(.) 연산자를 사용하여 액세스할 수 있습니다. 예: Myclass.Myattribute

클래스 만들기

여기서 class 키워드는 클래스를 생성한다는 것을 나타내며, 뒤에 클래스 이름(이 경우 Dog)이 붙습니다.

```
class Dog:
    species = "Canine"  # Class attribute

    def __init__(self, name, age):
        self.name = name  # Instance attribute
        self.age = age  # Instance attribute
```

설명:

- **클래스 Dog:** Dog라는 이름의 클래스를 정의합니다.
- **종:** 클래스의 모든 인스턴스가 공유하는 클래스 속성입니다.
- **__init__ 메서드:** 새 객체가 생성될 때 name 및 age 속성을 초기화합니다.

파이썬 객체

객체는 클래스의 인스턴스입니다. 클래스의 특정 구현을 나타내며 자체 데이터를 보유합니다.

객체는 다음으로 구성됩니다.

- **상태:** 속성으로 표현되며 객체의 속성을 반영합니다.
- **동작:** 객체의 메서드로 표현되며 객체의 다른 객체에 대한 응답을 반영합니다.
- **정체성:** 객체에 고유한 이름을 부여하고 한 객체가 다른 객체와 상호 작용할 수 있게 해줍니다.

객체 생성

파이썬에서 객체를 생성하려면 클래스를 인스턴스화하여 해당 클래스의 새 인스턴스를 만들어야 합니다. 이 과정을 객체 인스턴스화라고도 합니다.

```
class Dog:
    species = "Canine"  # Class attribute

    def __init__(self, name, age):
        self.name = name  # Instance attribute
        self.age = age  # Instance attribute

# Creating an object of the Dog class
dog1 = Dog("Buddy", 3)

print(dog1.name)
print(dog1.species)
```

- **dog1 = Dog("Buddy", 3):** 이름이 "Buddy"이고 나이가 3인 Dog 클래스의 객체를 생성합니다.
- **dog1.name:** dog1 객체의 인스턴스 속성 이름에 액세스합니다.
- **dog1.species:** dog1 객체의 클래스 속성 species에 접근합니다.

자기 매개변수

self 매개변수는 클래스의 현재 인스턴스에 대한 참조입니다. 이를 통해 객체의 속성과 메서드에 접근할 수 있습니다.

```
class Dog:
    species = "Canine"  # Class attribute

    def __init__(self, name, age):
        self.name = name  # Instance attribute
        self.age = age  # Instance attribute

dog1 = Dog("Buddy", 3)  # Create an instance of Dog
dog2 = Dog("Charlie", 5)  # Create another instance of Dog

print(dog1.name, dog1.age, dog1.species)  # Access instance and class
attributes
print(dog2.name, dog2.age, dog2.species)  # Access instance and class
attributes
print(Dog.species)  # Access class attribute directly
```

산출

버디 3 캐나인
찰리 5마리
송곳니

설명:

- **self.name:** 메서드를 호출하는 객체(dog1)의 이름 속성을 참조합니다.
- **dog1.bark():** dog1의 bark 메서드를 호출합니다.

__init__ 메서드

__init__ 메서드는 Python의 생성자로, 새 객체가 생성될 때 자동으로 호출됩니다. 클래스의 속성을 초기화합니다.

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

dog1 = Dog("Buddy", 3)
print(dog1.name)
```

설명:

- **__init__:** 초기화에 사용되는 특수 메서드.
- **self.name 및 self.age:** 생성자에서 초기화된 인스턴스 속성입니다.

클래스 및 인스턴스 변수

파이썬에서 클래스에 정의된 변수는 클래스 변수이거나 인스턴스 변수일 수 있으며, 객체 지향 프로그래밍을 위해서는 이 둘의 차이점을 이해하는 것이 중요합니다.

클래스 변수

클래스의 모든 인스턴스에서 공유되는 변수입니다. 클래스 수준에서 메서드 외부에서 정의됩니다. 클래스의 모든 객체는 객체에서 명시적으로 오버라이드되지 않는 한 클래스 변수에 대해 동일한 값을 공유합니다.

인스턴스 변수

클래스의 각 인스턴스(객체)에 고유한 변수입니다. 이러한 변수는 __init__ 메서드 또는 다른 인스턴스 메서드 내에서 정의됩니다. 각 객체는 다른 객체와 독립적으로 인스턴스 변수의 사본을 유지합니다.

```
class Dog:
    # Class variable
    species = "Canine"

    def __init__(self, name, age):
        # Instance variables
        self.name = name
        self.age = age

# Create objects
```

```
dog1 = Dog("Buddy", 3)
dog2 = Dog("Charlie", 5)

# Access class and instance variables
print(dog1.species)  # (Class variable)
print(dog1.name)     # (Instance variable)
print(dog2.name)     # (Instance variable)

# Modify instance variables
dog1.name = "Max"
print(dog1.name)     # (Updated instance variable)

# Modify class variable
Dog.species = "Feline"
print(dog1.species)  # (Updated class variable)
print(dog2.species)
```

산출

설명:

- **클래스 변수(종):** 클래스의 모든 인스턴스에서 공유됩니다. Dog.species를 변경하면 클래스 자체의 속성이므로 모든 객체에 영향을 미칩니다.
- **인스턴스 변수(이름, 나이):** __init__ 메서드에 정의됩니다. 각 인스턴스마다 고유합니다(예: dog1.name과 dog2.name은 서로 다릅니다).
- **변수 접근:** 클래스 변수는 클래스 이름(Dog.species) 또는 객체(dog1.species)를 통해 접근할 수 있습니다. 인스턴스 변수는 객체(dog1.name)를 통해 접근합니다.
- **변수 업데이트:** Dog.species를 변경하면 모든 인스턴스에 영향을 미칩니다. dog1.name을 변경하면 dog1에만 영향을 미치고 dog2에는 영향을 미치지 않습니다.

파이썬 상속

상속은 한 클래스(자식 클래스)가 다른 클래스(부모 클래스)의 속성과 메서드를 상속받을 수 있도록 합니다. 상속은 계층적 분류를 지원하고 코드 재사용을 촉진합니다.

상속의 종류:

1. **단일 상속:** 자식 클래스는 단일 부모 클래스로부터 상속받습니다.
2. **다중 상속:** 자식 클래스는 두 개 이상의 부모 클래스로부터 상속받습니다.
3. **다단계 상속:** 자식 클래스는 부모 클래스로부터 상속받고, 부모 클래스는 다시 다른 클래스로부터 상속받습니다.
4. **계층적 상속:** 여러 자식 클래스가 하나의 부모 클래스로부터 상속받습니다.
5. **혼합형 상속:** 두 가지 이상의 상속 유형을 결합한 형태입니다.

```
# Single Inheritance
class Dog:
    def __init__(self, name):
        self.name = name

    def display_name(self):
        print(f"Dog's Name: {self.name}")

class Labrador(Dog):  # Single Inheritance
    def sound(self):
        print("Labrador woofs")

# Multilevel Inheritance
class GuideDog(Labrador):  # Multilevel Inheritance
    def guide(self):
        print(f"{self.name}Guides the way!")

# Multiple Inheritance
class Friendly:
    def greet(self):
        print("Friendly!")

class GoldenRetriever(Dog, Friendly):  # Multiple Inheritance
    def sound(self):
        print("Golden Retriever Barks")

# Example Usage
lab = Labrador("Buddy")
lab.display_name()
lab.sound()

guide_dog = GuideDog("Max")
guide_dog.display_name()
guide_dog.guide()

retriever = GoldenRetriever("Charlie")
retriever.display_name()
retriever.greet()
retriever.sound()
```

설명:

- **단일 상속:** Labrador는 Dog의 속성과 메서드를 상속받습니다.
- **다중 레벨 상속:** GuideDog는 Labrador를 확장하여 Dog와 Labrador의 기능을 모두 상속합니다.
- **다중 상속:** GoldenRetriever는 Dog와 Friendly 모두로부터 상속을 받습니다.

파이썬 다형성

다형성은 메서드 이름이 같지만 객체의 컨텍스트에 따라 다르게 동작하도록 하는 것입니다. 메서드 오버라이딩이나 오버로딩을 통해 구현할 수 있습니다.

다형성의 유형

1. **컴파일 타임 다형성** : 이 유형의 다형성은 프로그램 컴파일 과정에서 결정됩니다. 같은 이름을 가진 메서드나 연산자가 입력 매개변수나 사용법에 따라 다르게 동작할 수 있도록 합니다. 일반적으로 메서드 또는 연산자 오버로딩이라고 합니다.
2. **런타임 다형성** : 이 유형의 다형성은 프로그램 실행 중에 결정됩니다. 하위 클래스가 부모 클래스에 이미 정의된 메서드를 특정 방식으로 구현할 때 발생하는데, 이를 메서드 오버라이딩이라고 합니다.

코드 예제:

```
# Parent Class
class Dog:
    def sound(self):
        print("dog sound")  # Default implementation

# Run-Time Polymorphism: Method Overriding
class Labrador(Dog):
    def sound(self):
        print("Labrador woofs")  # Overriding parent method

class Beagle(Dog):
    def sound(self):
        print("Beagle Barks")  # Overriding parent method

# Compile-Time Polymorphism: Method Overloading Mimic
class Calculator:
    def add(self, a, b=0, c=0):
        return a + b + c  # Supports multiple ways to call add()

# Run-Time Polymorphism
dogs = [Dog(), Labrador(), Beagle()]
for dog in dogs:
    dog.sound()  # Calls the appropriate method based on the object type


# Compile-Time Polymorphism (Mimicked using default arguments)
calc = Calculator()
print(calc.add(5, 10))  # Two arguments
print(calc.add(5, 10, 15))  # Three arguments
```

설명:

1. 런타임 다형성:

- Dog 클래스와 그 하위 클래스(Labrador 및 Beagle)에서 메서드 오버라이딩을 사용하는 방법을 보여줍니다.
- 올바른 사운드 메서드는 목록에 있는 객체의 실제 유형에 따라 런타임에 호출됩니다.

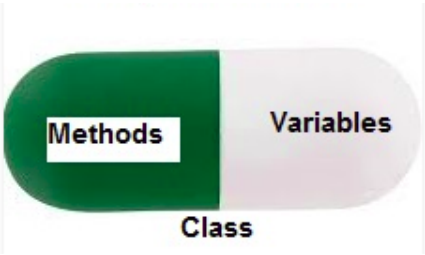
2. 컴파일 타임 다형성:

- Python은 기본적으로 메서드 오버로딩을 지원하지 않습니다. 대신, 기본 인수를 갖는 단일 메서드(add)를 사용하여 다양한 개수의 매개변수를 처리합니다.
- 메서드가 호출되는 방식에 따라 서로 다른 동작(두 개 또는 세 개의 숫자 추가)이 수행됩니다.

파이썬 캡슐화

캡슐화는 클래스 내에서 데이터(속성)와 메서드(함수)를 묶어서 일부 구성 요소에 대한 액세스를 제한하여 상호 작용을 제어하는 것입니다.

클래스는 멤버 함수, 변수 등 모든 데이터를 캡슐화하므로 캡슐화의 한 예입니다.



캡슐화 유형:

1. **공개 멤버** : 어디서나 접근 가능.
2. **보호된 멤버** : 클래스와 하위 클래스 내에서 접근 가능합니다.
3. **비공개 멤버** : 클래스 내에서만 접근 가능합니다.

코드 예제:

```
class Dog:
    def __init__(self, name, breed, age):
        self.name = name  # Public attribute
        self._breed = breed  # Protected attribute
        self.__age = age  # Private attribute
```

```
    # Public method
    def get_info(self):
        return f"Name: {self.name}, Breed: {self._breed}, Age: {self.__age}"

    # Getter and Setter for private attribute
    def get_age(self):
        return self.__age

    def set_age(self, age):
        if age > 0:
            self.__age = age
        else:
            print("Invalid age!")

# Example Usage
dog = Dog("Buddy", "Labrador", 3)

# Accessing public member
print(dog.name)  # Accessible

# Accessing protected member
print(dog._breed)  # Accessible but discouraged outside the class

# Accessing private member using getter
print(dog.get_age())

# Modifying private member using setter
dog.set_age(5)
print(dog.get_info())
```

설명:

- **공개 멤버:** 이름 등 쉽게 접근 가능.
- **보호된 멤버** : _breed와 같이 단일 _와 함께 사용됩니다. 접근은 권장되지 않지만 하위 클래스에서는 허용됩니다.
- **비공개 멤버:** __age와 같이 __와 함께 사용됩니다. 접근하려면 getter 및 setter 메서드가 필요합니다 .

데이터 추상화

추상화는 내부 구현 세부 사항을 숨기고 필요한 기능만 노출합니다. "어떻게 할 것인가"보다는 "무엇을 할 것인가"에 집중하는 데 도움이 됩니다.

추상화 유형:

- **부분적 추상화:** 추상 클래스는 추상 메서드와 구체적 메서드를 모두 포함합니다.
- **완전 추상화:** 추상 클래스는 추상 메서드(인터페이스와 같음)만 포함합니다.

코드 예제:

```python
from abc import ABC, abstractmethod

class Dog(ABC):  # Abstract Class
    def __init__(self, name):
        self.name = name

    @abstractmethod
    def sound(self):  # Abstract Method
        pass

    def display_name(self):  # Concrete Method
        print(f"Dog's Name: {self.name}")

class Labrador(Dog):  # Partial Abstraction
    def sound(self):
        print("Labrador Woof!")

class Beagle(Dog):  # Partial Abstraction
    def sound(self):
        print("Beagle Bark!")

# Example Usage
dogs = [Labrador("Buddy"), Beagle("Charlie")]
for dog in dogs:
    dog.display_name()  # Calls concrete method
    dog.sound()  # Calls implemented abstract method
```

설명:

- **부분 추상화** : `Dog` 클래스에는 추상(`sound`) 메서드와 구체적(`display_name`) 메서드가 모두 있습니다.
- **사용 이유** : 추상화는 추상 메서드의 구현을 강제함으로써 파생 클래스의 일관성을 보장합니다.

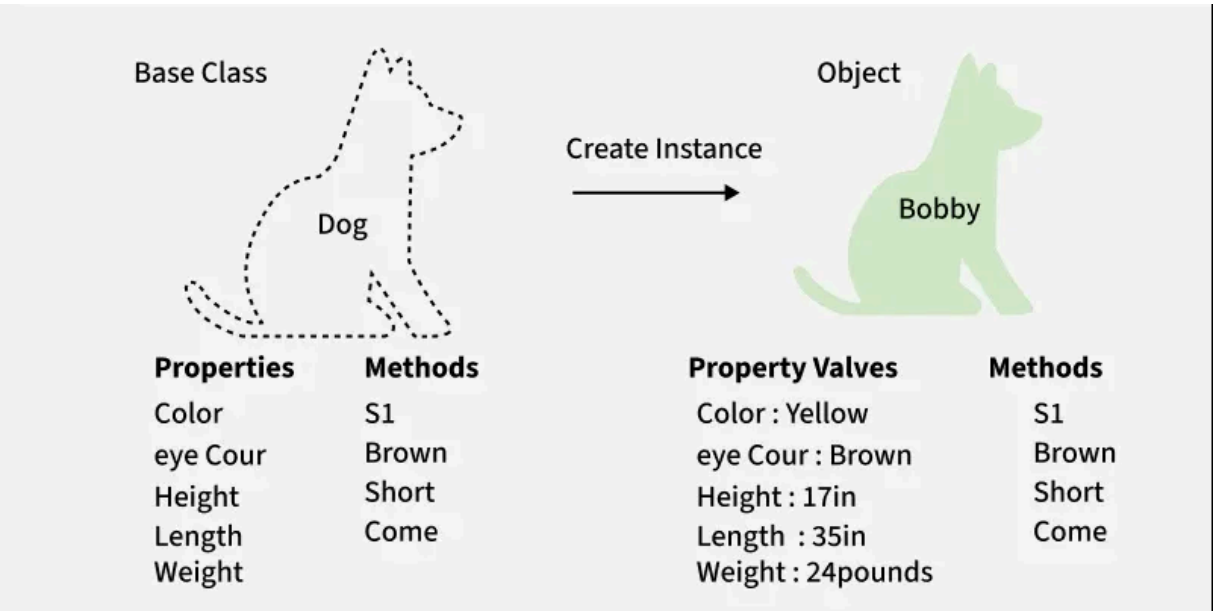
파이썬 클래스와 객체

Python에서 클래스와 객체는 객체 지향 프로그래밍 의 핵심입니다 . 클래스는 객체의 구조와 동작을 정의하고, 객체는 이러한 클래스의 인스턴스입니다. 클래스를 생성하고 사용하는 방법을 이해하는 것은 깔끔하고 유지 관리가 용이한 코드를 작성하는 데 필수적입니다. 이 글에서는 클래스 변수, 인스턴스 변수, 메서드 등의 핵심 개념을 다루어 Python의 객체 지향 접근 방식에 대한 탄탄한 기반을 다집니다.

클래스와 객체

클래스란 무엇인가요?

파이썬에서 클래스는 객체를 생성하기 위한 사용자 정의 템플릿입니다. 데이터와 함수를 하나로 묶어 관리하고 사용하기 쉽게 만듭니다. 새 클래스를 만들 때 새로운 유형의 객체를 정의합니다. 그러면 이 객체 유형의 인스턴스를 여러 개 만들 수 있습니다.



클래스는 class 키워드를 사용하여 생성됩니다 . 속성은 클래스 내부에 정의된 변수이며 클래스의 속성을 나타냅니다. 속성은 점 **(.) 연산자를** 사용하여 액세스할 수 있습니다 (예: MyClass.my_attribute).

```
# define a class
class Dog:
    sound = "bark"  # class attribute
```

객체란 무엇인가?

객체 **는** 클래스의 특정 인스턴스입니다. 자체 데이터 집합(인스턴스 변수)을 보유하고 클래스에 정의된 메서드를 호출할 수 있습니다. 동일한 클래스에서 고유한 속성을 가진 여러 객체를 생성할 수 있습니다.

Dog 클래스 에서 객체를 만들어 보겠습니다 .

```
class Dog:
    sound = "bark"


# Create an object from the class
dog1 = Dog()

# Access the class attribute
print(dog1.sound)
```

위 샘플 코드에서 **sound 속성은** 클래스 속성입니다. 이 속성은 Dog 클래스의 모든 인스턴스에서 공유되므로 **dog1** 인스턴스를 통해 직접 접근할 수 있습니다 .

파이썬 클래스와 객체가 필요한 이유

- 재사용 가능한 템플릿(클래스)과 실제 모델(객체)을 사용하여 객체 지향 프로그래밍을 지원합니다.
- 체계화된 메서드와 속성을 통해 코드 재사용성과 모듈형 디자인을 촉진합니다.
- 관련 데이터와 동작을 그룹화하여 복잡한 프로그램을 단순화합니다.
- 상속, 캡슐화, 다형성과 같은 핵심 OOP 개념을 지원합니다.

__init__() 함수 사용하기

Python에서 클래스에는 객체가 생성될 때 객체 속성을 자동으로 초기화하는 __init__() 함수가 있습니다. 이 메서드는 Python의 생성자입니다.`__init__()`

```
class Dog:
    species = "Canine"  # Class attribute

    def __init__(self, name, age):
        self.name = name  # Instance attribute
        self.age = age  # Instance attribute
```

설명 :

- **클래스 Dog:** Dog라는 이름의 클래스를 정의합니다.
- **종:** 클래스의 모든 인스턴스가 공유하는 클래스 속성입니다.
- **__init__ 메서드:** 새 객체가 생성될 때 name 및 age 속성을 초기화합니다.

__init__()로 객체 초기화

```
class Dog:
    species = "Canine"  # Class attribute

    def __init__(self, name, age):
        self.name = name  # Instance attribute
        self.age = age  # Instance attribute

# Creating an object of the Dog class
dog1 = Dog("Buddy", 3)

print(dog1.name)  # Output: Buddy
print(dog1.species)  # Output: Canine
```

설명:

- **dog1 = Dog("Buddy", 3):** 이름이 "Buddy"이고 나이가 3인 Dog 클래스의 객체를 생성합니다.
- **dog1.name:** dog1 객체의 인스턴스 속성 이름에 액세스합니다.

- **dog1.species:** dog1 객체의 클래스 속성 species에 접근합니다.

자기 매개변수

self 매개변수는 클래스의 현재 인스턴스에 대한 참조입니다. 이를 통해 객체의 속성과 메서드 에 접근할 수 있습니다.

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def bark(self):
        print(f"{self.name} is barking!")

# Creating an instance of Dog
dog1 = Dog("Buddy", 3)
dog1.bark()
```

설명:

- bark() 내부에서 self.name은 특정 개의 이름에 접근하여 출력합니다.
- dog1.bark()를 호출하면 Python은 자동으로 dog1을 self로 전달하여 해당 속성에 액세스할 수 있도록 합니다.

__str__() 메서드

Python의 __str__ 메서드를 사용하면 객체의 사용자 지정 문자열 표현을 정의할 수 있습니다. 기본적으로 객체를 출력하거나 str()을 사용하여 문자열로 변환할 때 Python은 기본 구현을 사용하며, <__main__.ClassName object at 0x00000123>과 같은 문자열을 반환합니다.

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return f"{self.name} is {self.age} years old."  # Correct: Returning a
string

dog1 = Dog("Buddy", 3)
dog2 = Dog("Charlie", 5)

print(dog1)
print(dog2)
```

설명:

- **__str__ 구현:** Dog 클래스의 메서드로 정의됩니다. self 매개변수를 사용하여 인스턴스의 속성(이름 및 나이)에 접근합니다.
- **읽기 가능한 출력:** print(dog1)을 호출하면 Python은 자동으로 __str__ 메서드를 사용하여 객체의 문자열 표현을 가져옵니다. __str__ 메서드가 없다면 print(dog1)을 호출하면 <__main__.Dog object at 0x00000123>과 같은 결과가 생성됩니다.

Python의 클래스 및 인스턴스 변수

파이썬에서 클래스에 정의된 변수는 클래스 변수 이거나 인스턴스 변수일 수 있으며, 객체 지향 프로그래밍을 위해서는 이 둘의 차이점을 이해하는 것이 중요합니다.

클래스 변수

클래스의 모든 인스턴스에서 공유되는 변수입니다. 클래스 수준에서 메서드 외부에서 정의됩니다. 클래스의 모든 객체는 객체에서 명시적으로 오버라이드되지 않는 한 클래스 변수에 대해 동일한 값을 공유합니다.

인스턴스 변수

클래스의 각 인스턴스(객체)에 고유한 변수입니다. 이러한 변수는 **__init__()** 메서드 또는 다른 인스턴스 메서드 내에서 정의됩니다. 각 객체는 다른 객체와 독립적으로 인스턴스 변수의 사본을 유지합니다.

예:

```
class Dog:
    # Class variable
    species = "Canine"

    def __init__(self, name, age):
        # Instance variables
        self.name = name
        self.age = age

# Create objects
dog1 = Dog("Buddy", 3)
dog2 = Dog("Charlie", 5)

# Access class and instance variables
print(dog1.species)  # (Class variable)
print(dog1.name)     # (Instance variable)
print(dog2.name)     # (Instance variable)
```

```python
# Modify instance variables
dog1.name = "Max"
print(dog1.name)     # (Updated instance variable)

# Modify class variable
Dog.species = "Feline"
print(dog1.species)  # (Updated class variable)
print(dog2.species)
```

설명:

- **클래스 변수(종):** 클래스의 모든 인스턴스에서 공유됩니다. Dog.species를 변경하면 클래스 자체의 속성이므로 모든 객체에 영향을 미칩니다.
- **인스턴스 변수(이름, 나이):** __init__() 메서드에 정의됩니다. 각 인스턴스마다 고유합니다(예: dog1.name과 dog2.name은 서로 다릅니다).
- **변수 접근:** 클래스 변수는 클래스 이름(Dog.species) 또는 객체(dog1.species)를 통해 접근할 수 있습니다. 인스턴스 변수는 객체(dog1.name)를 통해 접근합니다.
- **변수 업데이트:** Dog.species를 변경하면 모든 인스턴스에 영향을 미칩니다. dog1.name을 변경하면 dog1에만 영향을 미치고 dog2에는 영향을 미치지 않습니다.

Python 클래스 및 객체의 추가 중요 개념

게터 및 세터 메서드

Getter 및 **Setter** 메서드는 객체의 속성에 대한 제어된 접근을 제공합니다. Python에서 이러한 메서드는 비공개 속성의 값을 가져 `getter` 오거나(`setter`) 수정하는 데 사용되며, 이를 통해 데이터 캡슐화를 가능하게 합니다.

Python에는 다른 언어와 같이 명시 적인 메서드 가 없지만 **속성 데코레이터를** 사용하여 이 기능을 지원합니다 `.get set`

- **게터(Getter):** 속성의 값에 접근하는 데 사용됩니다.
- **세터** : 속성의 값을 수정하는 데 사용됩니다.

```python
class Dog:
    def __init__(self, name, age):
        self._name = name  # Conventionally private variable
        self._age = age  # Conventionally private variable

    @property
    def name(self):
        return self._name  # Getter
```

```
    @name.setter
    def name(self, value):
        self._name = value  # Setter

    @property
    def age(self):
        return self._age  # Getter

    @age.setter
    def age(self, value):
        if value < 0:
            print("Age cannot be negative!")
        else:
            self._age = value  # Setter
```

메서드 재정의

메서드 오버라이딩은 하위 클래스가 상위 클래스 에 이미 정의된 메서드의 특정 구현을 제공할 때 발생합니다 . 이를 통해 하위 클래스는 상속된 메서드의 동작을 수정하거나 확장할 수 있습니다.

```
class Animal:
    def sound(self):
        print("Some sound")


class Dog(Animal):
    def sound(self):  # Method overriding
        print("Woof")


dog = Dog()
dog.sound()  # Output: Woof
```

정적 메서드와 클래스 메서드

정적 메서드 와 **클래스 메서드는** 클래스 인스턴스가 아닌 클래스에 바인딩됩니다.

- **정적 메서드** : 인스턴스나 클래스에 접근할 필요가 없는 메서드입니다. `@staticmethod`데코레이터를 사용하여 정의됩니다.
- **클래스 메서드** : 클래스를 첫 번째 인수로 받는 메서드입니다(`cls`). 데코레이터를 사용하여 정의됩니다 `@classmethod`.

```
class Dog:
    @staticmethod
    def info():
        print("Dogs are loyal animals.")

    @classmethod
```

```
    def count(cls):
        print(f"There are many dogs of class {cls}.")


dog = Dog()
dog.info()  # Static method call
dog.count()  # Class method call
```

추상 클래스와 인터페이스

추상 클래스는 다른 클래스의 템플릿을 제공합니다. 이러한 클래스는 직접 인스턴스화할 수 없습니다. 추상 클래스에는 하위 클래스에서 구현해야 하는 메서드인 추상 메서드가 포함되어 있습니다.

추상 클래스 는 Python의 모듈을 사용하여 정의됩니다 `.abc`

```
from abc import ABC, abstractmethod


class Animal(ABC):
    @abstractmethod
    def sound(self):
        pass


class Dog(Animal):
    def sound(self):
        print("Woof")


dog = Dog()
dog.sound()  # Output: Woof
```

클래스 변수 대 인스턴스 변수

- **클래스 변수는** 클래스의 모든 인스턴스에서 공유됩니다. 클래스 변수는 클래스 내부에서 정의되지만 메서드 외부에 정의됩니다.
- **인스턴스 변수는** 각 인스턴스마다 고유하며 메서드에서 정의됩니다.`__init__()`

```
class Dog:
    species = "Canine"  # Class variable

    def __init__(self, name, age):
        self.name = name  # Instance variable
        self.age = age  # Instance variable


dog1 = Dog("Buddy", 3)
dog2 = Dog("Lucy", 2)
```

```
print(dog1.species)  # Accessing class variable
print(dog2.name)  # Accessing instance variable
```

```
class MyClass:
    def __init__(self, arg1, arg2):
        self.arg1 = arg1
        self.arg2 = arg2

    def some_method(self):
        # Method definition
        pass
```

질문: OOP에서 객체란 무엇인가요?

A: 객체 지향 프로그래밍(OOP)에서 객체는 클래스의 특정 인스턴스를 나타내는 유형적인 개체입니다. 클래스에 지정된 데이터(속성)와 동작(메서드)을 결합합니다.

질문: 클래스와 객체는 왜 필요한가요?

A: 클래스와 객체는 실제 개체와 추상적인 개념을 코드로 모델링하는 방법을 제공합니다. 코드 구성, 캡슐화(데이터 은닉), 상속(코드 재사용), 다형성 (메서드 오버라이딩)을 촉진하여 복잡한 시스템을 관리하고 확장하기 쉽게 만듭니다.

파이썬의 다형성

최종 업데이트: 2025년 7월 26일

다형성은 "다양한 형태"를 의미합니다. 이는 함수나 객체와 같은 엔티티가 상황에 따라 다양한 동작을 수행할 수 있는 능력을 의미합니다.

기술적으로, 파이썬에서 다형성은 동일한 메서드, 함수 또는 연산자가 작업하는 객체에 따라 다르게 동작하도록 합니다. 이는 코드를 더욱 유연하고 재사용 가능하게 만듭니다.

다형성이 왜 필요한가요?

- 다양한 클래스에서 일관된 인터페이스를 보장합니다.
- 동일한 메서드 호출에 대해 개체가 다르게 응답할 수 있도록 합니다.
- 특정 유형이 아닌 공유된 동작에 의존하여 느슨한 결합을 촉진합니다.
- 여러 유형에 걸쳐 작동하는 유연하고 재사용 가능한 코드를 작성할 수 있습니다.
- 테스트와 향후 코드 확장이 간소화됩니다.

다형성의 예

리모컨 하나로 TV, 에어컨, 음악 시스템 등 여러 기기를 조작할 수 있습니다. 전원 버튼을 누르면 각 기기가 다르게 반응합니다. TV가 켜지고, 에어컨이 냉각되기 시작하며, 음악 시스템이 음악을 재생합니다.

여기서 다형성이란 동일한 인터페이스(전원 버튼)이지만 장치(객체)에 따라 동작이 다르다는 것을 의미합니다.

다형성의 유형

파이썬에서 다형성은 동일한 메서드나 연산이 객체나 컨텍스트에 따라 다르게 동작하는 능력을 의미합니다. 다형성에는 주로 **컴파일 타임 다형성** 과 **런타임 다형성이** 포함됩니다 .

각 유형에 대해 자세히 살펴보겠습니다.

1. 컴파일 타임 다형성

컴파일 타임 다형성이란 일반적으로 메서드나 연산자 오버로딩을 통해 컴파일 중에 어떤 메서드나 연산을 실행할지 결정하는 것을 의미합니다.

Java나 C++ 같은 언어는 이를 지원합니다. 하지만 Python은 동적 타입 언어이기 때문에 컴파일 시점이 아닌 런타임에 메서드 호출을 처리합니다. 따라서 Python에서는 진정한 메서드 오버로딩을 지원하지 않지만, 기본 인수나 가변 인수를 사용하면 비슷한 동작을 구현할 수 있습니다.

예:

이 코드는 Python에서 기본 인수와 가변 길이 인수를 사용하여 메서드 오버로딩하는 방법을 보여줍니다. multiply() 메서드는 서로 다른 개수의 입력에 대해 동작하며, 컴파일 타임 다형성을 모방합니다.

```
class Calculator:
    def multiply(self, a=1, b=1, *args):
        result = a * b
        for num in args:
            result *= num
        return result

# Create object
calc = Calculator()

# Using default arguments
print(calc.multiply())
print(calc.multiply(4))

# Using multiple arguments
print(calc.multiply(2, 3))
print(calc.multiply(2, 3, 4))
```

산출

```
1
4
6
24
```

2. 런타임 다형성(재정의)

런타임 다형성이란, 프로그램이 실행되는 동안 해당 메서드를 호출하는 객체를 기반으로 메서드의 동작이 결정된다는 것을 의미합니다.

Python에서는 **메서드 오버라이딩** 을 통해 이를 구현합니다 . 자식 클래스는 부모 클래스에 이미 정의된 메서드의 자체 버전을 제공합니다. Python은 동적이므로 이를

지원하여 동일한 메서드 호출이 객체 유형에 따라 다르게 동작할 수 있습니다.

예:
이 코드는 메서드 오버라이딩을 사용하여 런타임 다형성을 보여줍니다. sound() 메서드는 기본 클래스 Animal에 정의되어 있으며, Dog와 Cat에서 오버라이딩됩니다. 런타임에는 객체의 클래스에 따라 적절한 메서드가 호출됩니다.

```
class Animal:
    def sound(self):
        return "Some generic sound"

class Dog(Animal):
    def sound(self):
        return "Bark"

class Cat(Animal):
    def sound(self):
        return "Meow"

# Polymorphic behavior
animals = [Dog(), Cat(), Animal()]
for animal in animals:
    print(animal.sound())
```

산출

설명: 여기서 사운드 메서드는 객체가 개, 고양이, 동물인지에 따라 다르게 동작하며, 이 결정은 런타임에 이루어집니다. 이러한 동적 특성은 Python을 런타임 다형성에 특히 강력하게 만듭니다.

내장 함수의 다형성

len() 과 **max()** 같은 Python 내장 함수는 다형성을 가지고 있어 다양한 데이터 유형을 처리하고 전달된 객체의 유형에 따라 결과를 반환합니다. 이는 동일한 함수 이름이라도 입력에 따라 동작이 달라지는 동적 특성을 잘 보여줍니다.

예:

이 코드는 동일한 함수 이름을 사용하면서도 문자열, 목록, 숫자, 문자를 다르게 처리하는 Python 내장 함수의 다형성을 보여줍니다.

```
print(len("Hello"))  # String length
print(len([1, 2, 3]))  # List length
```

```
print(max(1, 3, 2))  # Maximum of integers
print(max("a", "z", "m"))  # Maximum in strings
```

산출

함수의 다형성

파이썬에서 다형성은 필요한 동작을 지원하는 한 함수가 다양한 객체 타입을 허용할 수 있도록 합니다. 덕 타이핑을 사용하는 파이썬은 객체의 타입이 아닌 올바른 메서드가 있는지 여부에 초점을 맞춰 유연하고 재사용 가능한 코드를 구현합니다.

예:

이 코드는 perform_task() 함수가 .use () 메서드를 갖는 한 다양한 객체 유형(펜과 지우개)과 함께 작동하여 유연하고 재사용 가능한 함수 디자인을 보여주는 덕 타이핑을 사용하여 다형성을 보여줍니다.

```
class Pen:
    def use(self):
        return "Writing"

class Eraser:
    def use(self):
        return "Erasing"

def perform_task(tool):
    print(tool.use())

perform_task(Pen())
perform_task(Eraser())
```

산출

연산자의 다형성

파이썬에서는 같은 연산자(+)가 피연산자 타입에 따라 다른 작업을 수행할 수 있습니다. 이를 연산자 오버로딩이라고 합니다. 이러한 유연성은 파이썬 다형성의 핵심 요소입니다.

예:

이 코드는 + 연산자가 정수 추가, 문자열 연결, 목록 병합 등 데이터 유형에 따라 다르게 동작하는 것을 보여주는 연산자 다형성을 보여줍니다. 모두 동일한 연산자를 사용합니다.

```
print(5 + 10)  # Integer addition
print("Hello " + "World!")  # String concatenation
print([1, 2] + [3, 4])  # List concatenation
```

산출

파이썬에서의 상속

상속은 객체 지향 프로그래밍 (OOP) 의 기본 개념으로 , 클래스(자식 클래스 또는 파생 클래스라고 함)가 다른 클래스(부모 클래스 또는 기반 클래스라고 함)의 속성과 메서드를 상속받을 수 있도록 합니다. 이 글에서는 Python 에서 상속에 대해 살펴보겠습니다 .

상속을 위한 구문

```
class ParentClass:
    # Parent class code here
    pass
class ChildClass(ParentClass):
    # Child class code here
    pass
```

구문 설명

부모 클래스 :

- 이는 다른 클래스가 상속받는 기본 클래스입니다.
- 여기에는 자식 클래스가 재사용할 수 있는 속성과 메서드가 포함되어 있습니다.

자식 클래스:

- 이는 부모 클래스로부터 상속받은 파생 클래스입니다.
- 상속 구문은 클래스 ChildClass(ParentClass)입니다.
- 재정의하지 않는 한, 자식 클래스는 자동으로 부모 클래스의 모든 속성과 메서드를 가져옵니다.

파이썬에서 상속이 필요한 이유는 무엇입니까?

- 클래스 간에 속성과 메서드를 공유하여 코드 재사용성을 높입니다.
- 동물 → 개, 사람 → 직원과 같은 실제 계층 구조를 모델링합니다.

- 부모 클래스의 중앙 집중식 업데이트를 통해 유지관리를 간소화합니다.
- 사용자 정의된 하위 클래스 동작에 대한 메서드 재정의를 활성화합니다.
- 다형성을 사용하여 확장 가능하고 연장 가능한 디자인을 지원합니다.

부모 클래스 생성

객체 지향 프로그래밍에서 부모 클래스 (기본 클래스라고도 함)는 다른 클래스가 상속할 수 있는 공통 속성과 메서드를 정의합니다. 이러한 속성과 메서드는 자식 클래스의 기반이 됩니다. 상속을 통해 자식 클래스는 부모 클래스가 제공하는 기능에 접근하고 확장할 수 있습니다.

다음은 Person이 부모 클래스인 예입니다.

```python
# A Python program to demonstrate inheritance
class Person(object):

  # Constructor
  def __init__(self, name, id):
    self.name = name
    self.id = id

  # To check if this person is an employee
  def Display(self):
    print(self.name, self.id)


# Driver code
emp = Person("Elon", 102) # An Object of Person
emp.Display()
```

산출

```
엘론 102
```

설명

- Person 클래스에는 name과 id라는 두 가지 속성이 있습니다. 이 속성들은 클래스 객체가 생성될 때 설정됩니다.
- display 메서드는 사람의 이름과 ID를 출력합니다.

자식 클래스 생성

자식 클래스(서브클래스라고도 함)는 부모 클래스의 속성과 메서드를 상속받는 클래스입니다. 자식 클래스는 추가적인 속성과 메서드를 추가하거나, 부모 클래스에서 상속받은 속성과 메서드를 재정의할 수도 있습니다.

이 경우 Emp는 Person 클래스에서 상속받은 자식 클래스입니다.

```
class Emp(Person):

  def Print(self):
    print("Emp class called")

Emp_details = Emp("Mayank", 103)

# calling parent class function
Emp_details.Display()

# Calling child class function
Emp_details.Print()
```

설명:

- **Emp 클래스는** Person 클래스로부터 name 및 id 속성과 display 메서드를 상속받습니다.
- **Emp의 __init__ 메서드** 는 super().__init__(name, id)를 호출하여 Person 클래스의 생성자를 호출하고 상속된 속성을 초기화합니다.

super() 함수

super() 함수는 부모 클래스의 메서드를 호출하는 데 사용됩니다. 특히 자식 클래스의 __init__() 메서드에서 상속된 속성을 초기화하는 데 자주 사용됩니다. 이렇게 하면 자식 클래스에서 부모 클래스의 기능을 활용할 수 있습니다.

예:

```
# Parent Class: Person
class Person:
    def __init__(self, name, idnumber):
        self.name = name
        self.idnumber = idnumber

    def display(self):
        print(self.name)
        print(self.idnumber)

# Child Class: Employee
class Employee(Person):
    def __init__(self, name, idnumber, salary, post):
        super().__init__(name, idnumber)  # Using super() to call Person's
__init__()
        self.salary = salary
        self.post = post
```

설명:

- super() 함수는 Employee의 __init__() 메서드 내부에서 Person의 생성자를 호출하고 상속된 속성(이름과 ID 번호)을 초기화하는 데 사용됩니다.

- 이렇게 하면 자식 클래스의 코드를 다시 작성하지 않고도 부모 클래스의 기능을 재사용할 수 있습니다.

속성 추가

상속이 확립되면 부모 클래스와 자식 클래스 모두 각자의 속성을 가질 수 있습니다. 속성은 클래스에 속하며 데이터를 저장하는 데 사용되는 특성입니다.

예:

```
# Parent Class: Person
class Person:
    def __init__(self, name, idnumber):
        self.name = name
        self.idnumber = idnumber

    def display(self):
        print(self.name)
        print(self.idnumber)

# Child Class: Employee
class Employee(Person):
    def __init__(self, name, idnumber, salary, post):
        super().__init__(name, idnumber)
        self.salary = salary
        self.post = post
```

설명:

- Person 클래스에는 **name** 과 **idnumber** 속성이 있습니다 .
- 직원 클래스에 급여와 직책 속성이 추가됩니다.
- 속성은 객체가 생성될 때 초기화되며, Person 및 Employee와 관련된 구체적인 데이터를 나타냅니다.

파이썬 상속의 유형

1. **단일 상속** : 자식 클래스는 하나의 부모 클래스로부터 상속받습니다.
2. **다중 상속** : 자식 클래스는 두 개 이상의 부모 클래스로부터 상속받습니다.
3. **다중 레벨 상속** : 클래스는 다른 클래스에서 파생된 클래스에서 파생됩니다.
4. **계층적 상속** : 여러 클래스가 하나의 부모 클래스로부터 상속받습니다.
5. **혼합형 상속** : 두 가지 이상의 상속 유형을 결합한 형태입니다.

예:

```
# 1. Single Inheritance
class Person:
    def __init__(self, name):
        self.name = name
```

```python
class Employee(Person):  # Employee inherits from Person
    def __init__(self, name, salary):
        super().__init__(name)
        self.salary = salary

# 2. Multiple Inheritance
class Job:
    def __init__(self, salary):
        self.salary = salary

class EmployeePersonJob(Employee, Job):  # Inherits from both Employee and Job
    def __init__(self, name, salary):
        Employee.__init__(self, name, salary)  # Initialize Employee
        Job.__init__(self, salary)             # Initialize Job

# 3. Multilevel Inheritance
class Manager(EmployeePersonJob):  # Inherits from EmployeePersonJob
    def __init__(self, name, salary, department):
        EmployeePersonJob.__init__(self, name, salary)  # Explicitly initialize
EmployeePersonJob
        self.department = department

# 4. Hierarchical Inheritance
class AssistantManager(EmployeePersonJob):  # Inherits from EmployeePersonJob
    def __init__(self, name, salary, team_size):
        EmployeePersonJob.__init__(self, name, salary)  # Explicitly initialize
EmployeePersonJob
        self.team_size = team_size

# 5. Hybrid Inheritance (Multiple + Multilevel)
class SeniorManager(Manager, AssistantManager):  # Inherits from both Manager
and AssistantManager
    def __init__(self, name, salary, department, team_size):
        Manager.__init__(self, name, salary, department)        # Initialize
Manager
        AssistantManager.__init__(self, name, salary, team_size)  # Initialize
AssistantManager

# Creating objects to show inheritance

# Single Inheritance
emp = Employee("John", 40000)
print(emp.name, emp.salary)

# Multiple Inheritance
emp2 = EmployeePersonJob("Alice", 50000)
print(emp2.name, emp2.salary)

# Multilevel Inheritance
mgr = Manager("Bob", 60000, "HR")
print(mgr.name, mgr.salary, mgr.department)

# Hierarchical Inheritance
asst_mgr = AssistantManager("Charlie", 45000, 10)
print(asst_mgr.name, asst_mgr.salary, asst_mgr.team_size)

# Hybrid Inheritance
sen_mgr = SeniorManager("David", 70000, "Finance", 20)
print(sen_mgr.name, sen_mgr.salary, sen_mgr.department, sen_mgr.team_size)
```

산출

```
요한복음 40000
앨리스 50000
밥 60000 HR
찰리 45000 10
데이비드 70000 금융 20
```

설명:

1. **단일 상속:** Employee는 Person으로부터 상속하며 급여 속성을 추가합니다.
2. **다중 상속:** EmployeePersonJob은 Employee와 Job으로부터 상속을 받아 이름과 급여에 모두 액세스할 수 있습니다.
3. **다중 수준 상속:** Manager는 이미 Employee와 Job을 포함하고 있는 EmployeePersonJob으로부터 상속받습니다.
4. **계층적 상속** : AssistantManager도 EmployeePersonJob으로부터 상속받으며, 이는 여러 자식 클래스가 동일한 부모로부터 상속받는 것을 보여줍니다.
5. **하이브리드 상속** : SeniorManager는 Manager(다중 레벨)와 AssistantManager(계층적)로부터 상속받으며 두 가지 상속 유형을 결합합니다.

파이썬의 추상 클래스

파이썬에서 추상 클래스는 스스로 인스턴스화할 수 없고 다른 클래스의 청사진 역할을 하도록 설계된 클래스입니다. 추상 클래스를 사용하면 하위 클래스에서 구현해야 하는 메서드를 정의할 수 있으므로, 일관된 인터페이스를 보장하는 동시에 하위 클래스에서 특정 구현을 제공할 수 있습니다.

파이썬의 추상 기본 클래스

ABC는 하위 클래스에서 구현해야 하는 메서드를 정의하여 하위 클래스가 일관된 구조를 따르도록 합니다. ABC를 사용하면 다양한 하위 클래스가 구현할 수 있는 공통 인터페이스를 정의하는 동시에 일정 수준의 추상화를 적용할 수 있습니다.

Python은 ABC를 정의하고 하위 클래스에서 추상 메서드 구현을 강제하기 위해 abc 모듈을 제공합니다.

예:

```
from abc import ABC, abstractmethod

# Define an abstract class
class Animal(ABC):

    @abstractmethod
    def sound(self):
        pass  # This is an abstract method, no implementation here.

# Concrete subclass of Animal
class Dog(Animal):

    def sound(self):
        return "Bark"  # Providing the implementation of the abstract method

# Create an instance of Dog
dog = Dog()
print(dog.sound())  # Output: Bark
```

코드 설명:

- **추상 기반 클래스:** Animal은 ABC(추상 기반 클래스)를 상속하는 추상 클래스입니다. 이 클래스는 sound()라는 추상 메서드를 포함하고 있기 때문에 직접 인스턴스화할 수 없습니다. @abstractmethod 데코레이터는 sound()를 추상 메서드로

표시하는 데 사용됩니다. 즉, 모든 하위 클래스는 이 메서드를 구현해야 인스턴스화할 수 있습니다.

- **구체적 하위 클래스:** Dog는 Animal의 하위 클래스로, sound() 메서드를 구현합니다. 이를 통해 Dog 클래스를 인스턴스화하고 사용할 수 있습니다.
- **인스턴스화:** Dog의 인스턴스를 생성하고 sound() 메서드를 호출합니다. 이 메서드는 "Bark"를 반환합니다.

추상 메서드

추상 메서드는 추상 클래스에 정의되어 있지만 구현이 없는 메서드입니다. 추상 메서드는 하위 클래스의 청사진 역할을 하며, 하위 클래스가 자체 구현을 제공하도록 보장합니다.

예:

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def make_sound(self):
        pass  # Abstract method, no implementation here
```

설명:

- make_sound()는 Animal 클래스의 추상 메서드이므로 내부에 코드가 없습니다.

구체적인 방법

구체적 메서드는 추상 클래스에서 완전한 구현을 갖는 메서드입니다. 이러한 메서드는 하위 클래스에서 상속받아 재정의할 필요 없이 바로 사용할 수 있습니다.

예:

```
from abc import ABC, abstractmethod
```

```
class Animal(ABC):
    @abstractmethod
    def make_sound(self):
        pass  # Abstract method, to be implemented by subclasses

    def move(self):
        return "Moving"  # Concrete method with implementation
```

설명:

- move() 메서드는 Animal 클래스의 구체적 메서드입니다. Dog 클래스에서 구현되어 있으므로 오버라이드할 필요가 없습니다.

추상 속성

추상 속성은 추상 메서드처럼 동작하지만 속성에 사용됩니다. 이러한 속성은 `@property`데코레이터로 선언되고 . 를 사용하여 추상으로 표시됩니다 `@abstractmethod`. 하위 클래스는 이러한 속성을 구현해야 합니다.

예:

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @property
    @abstractmethod
    def species(self):
        pass  # Abstract property, must be implemented by subclasses

class Dog(Animal):
    @property
    def species(self):
        return "Canine"

# Instantiate the concrete subclass
dog = Dog()
print(dog.species)
```

설명:

- species는 Animal 클래스의 추상 속성이며 @abstractmethod로 표시됩니다.
- Dog 클래스는 종 속성을 구현하여 인스턴스화할 수 있는 구체적 하위 클래스를 만듭니다.
- 추상 속성은 하위 클래스가 속성의 구현을 제공하도록 강제합니다.

추상 클래스 인스턴스화

추상 클래스는 직접 인스턴스화할 수 없습니다. 구현이 없는 하나 이상의 추상 메서드나 속성을 포함하고 있기 때문입니다. 추상 클래스를 인스턴스화하려고 하면 TypeError가 발생합니다.

예:

```python
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def make_sound(self):
        pass

# Trying to instantiate the abstract class directly
# This will raise an error:
# animal = Animal()  # TypeError: Can't instantiate abstract class Animal with abstract methods make_sound
```

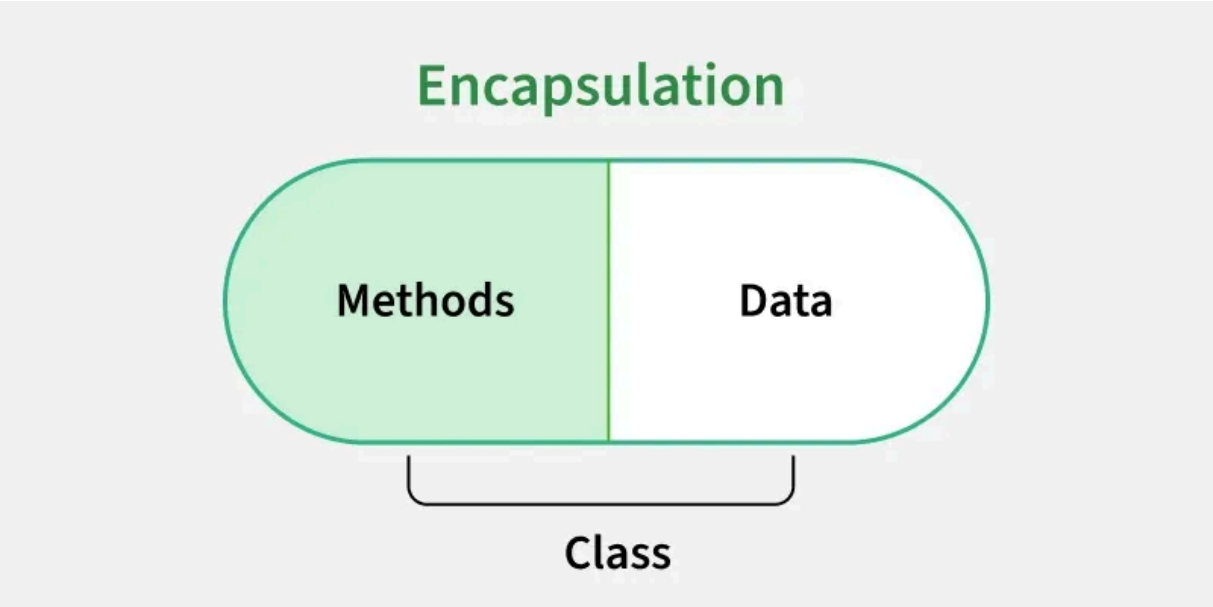
설명:

- Animal 클래스는 make_sound() 메서드를 추상 메서드로 가지고 있기 때문에 추상적입니다.
- Animal을 직접 인스턴스화하려고 하면(예: animal = Animal()), Python은 구현되지 않은 추상 메서드가 있는 클래스를 인스턴스화할 수 없기 때문에 TypeError를 발생시킵니다.
- 모든 추상 메서드를 구현하는 하위 클래스만 인스턴스화될 수 있습니다.

파이썬에서의 캡슐화

캡슐화는 클래스의 내부 세부 정보를 숨기고 필요한 부분만 노출하는 것을 의미합니다. 중요한 데이터가 직접 변경되는 것을 방지하고 코드를 안전하고 체계적으로 유지하는 데 도움이 됩니다.

기술적으로 캡슐화는 데이터(변수)와 메서드(함수)가 클래스로 묶여 있는 객체 지향 프로그래밍 원칙입니다.



캡슐화

캡슐화가 필요한 이유는 무엇인가?

- 데이터를 무단 접근 및 실수로 인한 수정으로부터 보호합니다.
- 검증을 통해 getter/setter 메서드를 사용하여 데이터 업데이트를 제어합니다.
- 내부 구현 세부 정보를 숨겨 모듈성을 향상시킵니다.
- 중앙 집중화된 데이터 처리 논리를 통해 유지관리를 간소화합니다.
- 은행 계좌 잔액에 대한 직접 접근을 제한하는 것과 같은 실제 시나리오를 반영합니다.

캡슐화의 예

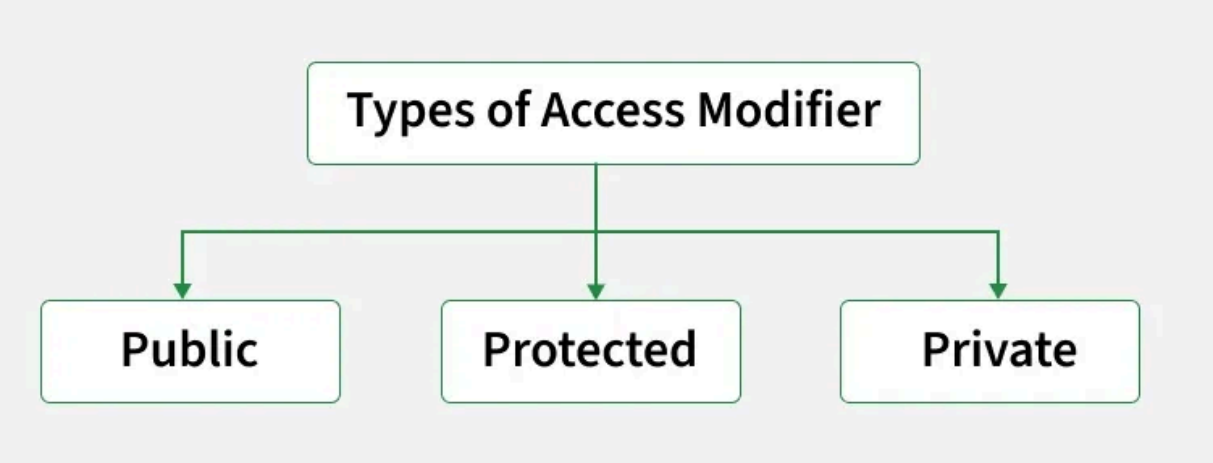
파이썬 캡슐화는 계좌 잔액(데이터)이 비공개로 유지되는 은행 계좌 시스템과 같습니다. 계좌 데이터베이스에 접근하여 잔액을 직접 변경할 수는 없습니다. 대신, 은행은

예금(deposit) 및 인출(withdraw)과 같은 메서드(함수)를 제공하여 잔액을 안전하게 변경할 수 있도록 합니다.

- **개인 정보(잔액)** : 잔액은 안전하게 저장됩니다. 외부에서의 직접 접근은 허용되지 않으므로 무단 변경으로부터 데이터가 안전하게 보호됩니다.
- **공개 방식(입금 및 출금)** : 잔액을 변경할 수 있는 유일한 방법입니다. 이 방식은 변경을 허용하기 전에 요청(출금 등)이 규칙(예: 잔액이 충분한지)을 준수하는지 확인합니다.

Python의 액세스 지정자

접근 지정자는 클래스 외부에서 클래스 멤버(변수와 메서드)에 접근하는 방법을 정의합니다. 접근 지정자는 데이터의 가시성을 제어하여 캡슐화를 구현하는 데 도움을 줍니다. 접근 지정자에는 세 가지 유형이 있습니다.



액세스 수정자 유형

하나하나씩 논의해 보겠습니다.

공공 회원

공개 멤버는 클래스 내부, 외부 또는 다른 모듈 어디에서나 접근할 수 있는 변수나 메서드입니다. 기본적으로 Python의 모든 멤버는 공개되어 있습니다.

이러한 이름은 밑줄 접두사 없이 정의됩니다(예: self.name).

예:

이 예제는 객체를 사용하여 클래스 외부에서 **공개 속성(이름)** 과 **공개 메서드(디스플레이_이름)에 액세스하는 방법을 보여줍니다.**

```
class Public:
    def __init__(self):
        self.name = "John"  # Public attribute
```

```
    def display_name(self):
        print(self.name)  # Public method

obj = Public()
obj.display_name()  # Accessible
print(obj.name)  # Accessible
```

산출

```
남자
남자
```

설명:

- **공개 속성(이름):** 이 속성은 밑줄 접두사 없이 선언됩니다. 클래스 내부와 외부 어디에서나 접근할 수 있습니다.
- **공개 메서드(display_name):** 이 메서드는 코드의 어느 부분에서나 접근할 수 있습니다. 공개 속성에 직접 접근하여 값을 출력합니다.
- **객체(obj):** Public 인스턴스가 생성되고 display_name 메서드가 호출되어 공개 속성과 메서드에 직접 액세스하는 방법을 보여줍니다.

> ***참고:*** *__init__ 메서드 는 생성자이며 클래스의 객체가 인스턴스화되면 즉시 실행됩니다.*

보호된 회원

보호된 멤버는 해당 클래스와 그 하위 클래스 내에서만 접근하도록 의도된 변수 또는 메서드입니다. 엄격하게 private은 아니지만, 내부적으로 처리해야 합니다.

Python에서 보호된 멤버는 단일 밑줄 접두사로 정의됩니다(예: self._name).

예:

이 예제는 보호된 속성 **(_age)** 이 하위 클래스 내에서 어떻게 액세스될 수 있는지를 보여주며, 보호된 멤버가 클래스와 그 하위 클래스 내에서 사용되도록 되어 있음을 보여줍니다.

```
class Protected:
    def __init__(self):
        self._age = 30  # Protected attribute

class Subclass(Protected):
    def display_age(self):
        print(self._age)  # Accessible in subclass
```

```
obj = Subclass()
obj.display_age()
```

산출

```
30
```

설명:

- **보호된 속성(_age):** 보호됨을 나타내기 위해 밑줄 하나를 접두사로 붙입니다. 이는 Python의 규칙이며, 강제되지 않으면 제한된 접근 권한을 나타냅니다.
- **하위 클래스 접근:** 하위 클래스는 Protected로부터 상속받으며 _age에 직접 접근할 수 있습니다.
- **메서드(display_age):** 보호된 멤버가 클래스와 하위 클래스 내에서 접근 가능함을 보여줍니다.

개인 회원

프라이빗 멤버는 클래스 외부에서 직접 접근할 수 없는 변수나 메서드입니다. 접근을 제한하고 내부 데이터를 보호하는 데 사용됩니다.

Python에서 비공개 멤버는 두 개의 밑줄 접두사로 정의됩니다(예: self.__salary). Python은 내부적으로 멤버의 이름을 변경하여 이름 맹글링을 적용합니다 (예: __salary를 _ClassName__salary로 변경). 이렇게 하면 멤버에 직접 액세스하는 것을 방지할 수 있습니다.

예:

이 예제는 공개 메서드를 사용하여 클래스 내에서 비공개 속성 **(__salary)** 에 액세스하는 방법을 보여주며, 비공개 멤버는 클래스 외부에서 직접 액세스할 수 없음을 보여줍니다.

```
class Private:
    def __init__(self):
        self.__salary = 50000  # Private attribute

    def salary(self):
        return self.__salary  # Access through public method

obj = Private()
print(obj.salary())  # Works
#print(obj.__salary)  # Raises AttributeError
```

산출

설명:

- **비공개 속성(__salary):** 비공개로 설정하려면 밑줄 두 개를 사용합니다. Python은 이름 변경을 사용하여 직접 액세스를 제한합니다.
- **메서드(salary):** 비공개 속성의 값을 안전하게 반환하는 공개 메서드입니다.
- **직접 접근:** obj.__salary를 시도하면 AttributeError가 발생하여, 비공개 멤버에 직접 접근할 수 없음을 보여줍니다.

보호된 메서드와 개인 메서드 선언

Python에서는 명명 규칙을 사용하여 메서드 액세스 수준을 제어할 수 있습니다.

- 메서드 이름 앞에 밑줄(_) 하나를 사용하여 해당 메서드가 클래스나 하위 클래스 내에서 사용하도록 보호된다는 것을 나타냅니다.
- 이름 변형으로 인해 클래스 내에서만 접근 가능한 개인 메서드를 정의하려면 이중 밑줄(__)을 사용합니다.

> ***참고:*** *다른 프로그래밍 언어와 달리 Python은 언어 수준에서 public, private, protected와 같은 접근 제한자를 강제하지 않습니다. 하지만 명명 규칙을 따르고* ***이름 맹글링(name mangling)*** *이라는 기법을 사용하여 캡슐화를 지원합니다.*

예:

이 예제는 **보호된 메서드(_show_balance)** 와 **개인 메서드(__update_balance)를** 사용하여 액세스를 제어하는 방법을 보여줍니다. 개인 메서드는 내부적으로 잔액을 업데이트하는 반면, 보호된 메서드는 잔액을 표시합니다. 두 메서드 모두 공개 메서드(deposit)를 통해 액세스하며, 이는 Python이 캡슐화를 위해 명명 규칙을 사용하는 방식을 보여줍니다.

```
class BankAccount:
    def __init__(self):
        self.balance = 1000

    def _show_balance(self):
        print(f"Balance: ₹{self.balance}")  # Protected method

    def __update_balance(self, amount):
        self.balance += amount              # Private method

    def deposit(self, amount):
        if amount > 0:
            self.__update_balance(amount)  # Accessing private method
```

```
internally
            self._show_balance()           # Accessing protected method
        else:
            print("Invalid deposit amount!")

account = BankAccount()
account._show_balance()      # Works, but should be treated as internal
# account.__update_balance(500)  # Error: private method
account.deposit(500)         # Uses both methods internally
```

산출

설명:

- **_show_balance():** (보호된 메서드) 외부에서 접근 가능하지만 내부 또는 하위 클래스에서 사용하도록 의도되었습니다.
- **__update_balance():** (비공개 메서드) 이름 변형으로 인해 클래스 내부에서만 접근 가능합니다.
- **deposit():** 비공개 메서드와 보호된 메서드를 모두 안전하게 사용하는 공개 메서드입니다.

게터 및 세터 메서드

Python에서는 **getter** 및 **setter** 메서드를 사용하여 비공개 속성에 안전하게 접근하고 수정할 수 있습니다. 비공개 데이터에 직접 접근하는 대신, 이러한 메서드는 제어된 접근을 제공하여 다음과 같은 작업을 수행할 수 있습니다.

- 게터 메서드를 사용하여 데이터를 읽습니다.
- 선택적 검증이나 제한 사항이 있는 세터 메서드를 사용하여 데이터를 업데이트합니다.

예:

이 예제에서는 게터와 세터 메서드를 사용하여 개인 속성(__salary)에 안전하게 액세스하고 업데이트하는 방법을 보여줍니다.

```
class Employee:
    def __init__(self):
        self.__salary = 50000  # Private attribute

    def get_salary(self):    # Getter method
        return self.__salary
```

```
    def set_salary(self, amount):   # Setter method
        if amount > 0:
            self.__salary = amount
        else:
            print("Invalid salary amount!")

emp = Employee()
print(emp.get_salary())  # Access salary using getter

emp.set_salary(60000)   # Update salary using setter
print(emp.get_salary())
```

산출

설명:

- **__salary** 는 비공개 속성이므로 클래스 외부에서 직접 액세스할 수 없습니다.
- **get_salary()** 는 현재 급여를 안전하게 반환하는 게터 메서드입니다.
- **set_salary(amount)** 는 금액이 양수일 때만 급여를 업데이트하는 세터 메서드입니다.
- 객체 **emp는** 이러한 메서드를 사용하여 데이터를 보호하면서 급여에 액세스하고 이를 수정합니다.